常见 Vite 配置详解

一、配置文件基础

1. 配置文件位置与格式

Vite 配置文件位于项目根目录，支持多种格式：

- `vite.config.js` — CommonJS 或 ESM
- `vite.config.ts` — TypeScript（推荐）
- `vite.config.mjs` — 强制 ESM
- `vite.config.cjs` — 强制 CommonJS

2. 基本结构

```
// 方式一：直接导出对象
export default {
  // 配置项
}

// 方式二：使用 defineConfig（推荐，有类型提示）
import { defineConfig } from 'vite'
export default defineConfig({
  // 配置项
})

// 方式三：函数式（可获取 mode、command 等环境信息）
import { defineConfig } from 'vite'
export default defineConfig(({ command, mode, isSsrBuild, isPreview }) => {
  // command: 'serve'（开发）或 'build'（生产）
  // mode: 'development' 或 'production' 或自定义
  return {
    // 根据环境返回不同配置
  }
})

```

二、核心配置项详解

1. `root` — 项目根目录

```
{
  root: process.cwd(), // 默认值，通常不需要修改
}

```

- 指定项目根目录（`index.html` 所在位置）
- 可以是绝对路径或相对于配置文件的路径

---

2. `base` — 公共基础路径

```
{
  base: '/',            // 默认值，部署在域名根路径
  base: '/admin/',      // 部署在子路径
  base: './',           // 相对路径（用于 file:// 协议）
  base: 'https://cdn.example.com/', // CDN 地址
}

```

面试考点：

- 影响所有静态资源的引用路径
- 类似 webpack 的 `publicPath`
- 部署到非根路径时必须配置

---

3. `mode` — 模式

```
{
  mode: 'development', // 开发模式
  mode: 'production',  // 生产模式
  mode: 'staging',     // 自定义模式
}
```

```

```

- 通过 `--mode` 命令行参数指定：`vite build --mode staging`
- 影响 `import.meta.env.MODE` 的值
- 决定加载哪个 `.env` 文件（如 `.env.staging`）

---

4. `define` — 全局常量替换

```
{
  define: {
    __APP_VERSION__: JSON.stringify('1.0.0'),
    __BUILD_TIME__: JSON.stringify(new Date().toISOString()),
    'process.env.NODE_ENV': JSON.stringify('production'),
  },
}

```

注意事项：

- 值会被**直接文本替换**到代码中
- 字符串值必须用 `JSON.stringify()` 包裹
- 类似 webpack 的 `DefinePlugin`

---

5. `plugins` — 插件配置

```
import vue from '@vitejs/plugin-vue'
import vueJsx from '@vitejs/plugin-vue-jsx'
import legacy from '@vitejs/plugin-legacy'

{
  plugins: [
    vue(),
    vueJsx(),
    legacy({
      targets: ['defaults', 'not IE 11'],
    }),
  ],
```

```
}

```

常用插件分类：

类别	插件	用途
框架	`@vitejs/plugin-vue`	Vue 3 支持
框架	`@vitejs/plugin-react`	React 支持
兼容	`@vitejs/plugin-legacy`	旧浏览器兼容
自动导入	`unplugin-auto-import`	API 自动导入
自动导入	`unplugin-vue-components`	组件自动注册
压缩	`vite-plugin-compression`	Gzip/Brotli
图标	`vite-plugin-svg-icons`	SVG 雪碧图
Mock	`vite-plugin-mock`	本地 Mock 数据
分析	`rollup-plugin-visualizer`	打包体积分析

---

6. `resolve` — 模块解析

```
{
  resolve: {
    // 路径别名
    alias: {
      '@': '/src',
      '@components': '/src/components',
      '@utils': '/src/utils',
      // 也可以用数组形式（支持正则）
    },

    // 导入时可省略的扩展名
    extensions: ['.mjs', '.js', '.ts', '.jsx', '.tsx', '.json', '.vue'],

    // 情景导出（package.json 的 exports 字段）
```

```
    conditions: ['import', 'module', 'browser', 'default'],

    // 解析 package.json 中的哪些字段
    mainFields: ['browser', 'module', 'jsdoc:main', 'main'],

    // 是否允许解析到软链接的真实路径
    preserveSymlinks: false,
  },
}

```

面试考点 — 路径别名配置：

```
// 方式一：对象形式
alias: {
  '@': path.resolve(__dirname, 'src'),
}

// 方式二：数组形式（支持正则匹配）
alias: [
  { find: '@', replacement: path.resolve(__dirname, 'src') },
  { find: /^~/, replacement: '' },
]

// 方式三：使用 fileURLToPath（ESM 推荐）
import { fileURLToPath, URL } from 'node:url'
alias: {
  '@': fileURLToPath(new URL('./src', import.meta.url)),
}

```

---

7. `css` — CSS 相关配置

```
{
  css: {
    // CSS Modules 配置
    modules: {
```

```
      localsConvention: 'camelCaseOnly', // 类名转换：camelCase / camelCaseOnly / dashes / dashesOnly
      scopeBehaviour: 'local',           // 'local' | 'global'
      generateScopedName: '[name]__[local]___[hash:base64:5]',
    },

    // PostCSS 配置（也可以用 postcss.config.js）
    postcss: {
      plugins: [
        require('autoprefixer'),
        require('postcss-pxtorem')({
          rootValue: 37.5,
          propList: ['*'],
        }),
      ],
    },

    // 预处理器配置
    preprocessorOptions: {
      scss: {
        additionalData: `@use "@/styles/variables.scss" as *;`, // 全局注入
        api: 'modern-compiler', // Dart Sass 新 API，更快
      },
      less: {
        additionalData: `@import "@/styles/variables.less";`,
        javascriptEnabled: true, // Ant Design 需要
        modifyVars: {
          'primary-color': '#1890ff',
        },
      },
    },

    // 开发时生成 sourcemap
    devSourcemap: true,
  },
}

```

面试考点 — 全局样式注入：

- `additionalData` ：在每个 SCSS/Less 文件开头自动注入内容
- 适合注入全局变量、混入（mixin）
- 注意：注入的是**代码字符串**，不是文件路径

8. `server` — 开发服务器配置

```
{
  server: {
    host: '0.0.0.0',        // 监听所有地址（局域网可访问）
    port: 5173,             // 端口
    strictPort: false,      // 端口被占用时是否直接退出
    open: true,             // 启动时自动打开浏览器
    open: '/login',         // 打开指定路径

    // HTTPS 配置
    https: {
      key: fs.readFileSync('path/to/key.pem'),
      cert: fs.readFileSync('path/to/cert.pem'),
    },

    // 代理配置（重点！）
    proxy: {
      // 字符串简写
      '/foo': 'http://localhost:4567',

      // 完整配置
      '/api': {
        target: 'http://backend.example.com',
        changeOrigin: true,                     // 修改请求头中的 Origin
        rewrite: (path) => path.replace(/^\/api/, ''),
        secure: false,                          // 允许无效证书
        ws: true,                               // 代理 WebSocket
        configure: (proxy, options) => {        // 自定义代理行为
          // proxy: http-proxy 实例
        },
      },

      // 正则匹配
      '^/fallback/.*': {
        target: 'http://jsonplaceholder.typicode.com',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/fallback/, ''),
      },
```

```
    },

    // CORS 配置
    cors: true,

    // 自定义响应头
    headers: {
      'Access-Control-Allow-Origin': '*',
    },

    // HMR 配置
    hmr: {
      overlay: true,      // 错误遮罩层
      port: 24678,        // HMR WebSocket 端口
    },

    // 文件监听配置
    watch: {
      usePolling: true,   // Docker/WSL 环境可能需要
      interval: 100,
    },

    // 预热常用文件（加速首次访问）
    warmup: {
      clientFiles: ['./src/components/*.vue'],
    },
  },
}

```

面试考点 — 代理配置：

- `target` ：目标服务器地址
- `changeOrigin: true` ：把请求头的 `Host` 改成 target 的地址（解决跨域）
- `rewrite` ：路径重写，去掉 `/api` 前缀
- 本质是用 `http-proxy` 实现的正向代理

---

9. `build` — 构建配置

```
{
```

```
  build: {
    outDir: 'dist',                    // 输出目录
    assetsDir: 'static',               // 静态资源目录（相对 outDir）
    assetsInlineLimit: 4096,           // 小于此值的资源内联为 base64（字节）

    // 构建目标
    target: 'es2020',                  // 或 ['es2020', 'edge88', 'firefox78', 'chrome87', 'safari14']

    // 压缩方式
    minify: 'esbuild',                 // 'esbuild'（快）| 'terser'（更小）| false
    terserOptions: {                   // minify 为 terser 时有效
      compress: {
        drop_console: true,
        drop_debugger: true,
      },
    },

    // Sourcemap
    sourcemap: false,                  // true | false | 'inline' | 'hidden'

    // CSS 代码拆分
    cssCodeSplit: true,                // true: 按 chunk 拆分 CSS

    // 清空输出目录
    emptyOutDir: true,

    // Chunk 大小警告阈值（KB）
    chunkSizeWarningLimit: 500,

    // Rollup 配置
    rollupOptions: {
      // 多入口
      input: {
        main: 'index.html',
        admin: 'admin.html',
      },

      // 外部化依赖（不打包）
      external: ['vue', 'lodash'],

      // 输出配置
```

```
43      output: {
44        // 手动分包
45        manualChunks: {
46          'vendor-vue': ['vue', 'vue-router', 'pinia'],
47          'vendor-ui': ['element-plus'],
48          'vendor-utils': ['axios', 'lodash-es', 'dayjs'],
49        },
50
51        // 或者使用函数形式（更灵活）
52        manualChunks(id) {
53          if (id.includes('node_modules')) {
54            if (id.includes('vue')) return 'vendor-vue'
55            if (id.includes('element-plus')) return 'vendor-ui'
56            return 'vendor'
57          }
58        },
59
60        // 文件命名
61        chunkFileNames: 'js/[name]-[hash].js',
62        entryFileNames: 'js/[name]-[hash].js',
63        assetFileNames: (assetInfo) => {
64          const ext = assetInfo.name?.split('.').pop() || ''
65          if (/png|jpe?g|gif|svg|webp|ico/i.test(ext)) {
66            return 'images/[name]-[hash][extname]'
67          }
68          if (/woff2?|eot|ttf|otf/i.test(ext)) {
69            return 'fonts/[name]-[hash][extname]'
70          }
71          return 'assets/[name]-[hash][extname]'
72        },
73
74        // 全局变量（配合 external 使用）
75        globals: {
76          vue: 'Vue',
77          lodash: '_',
78        },
79      },
80    },
81
82    // 库模式构建
83    lib: {
84      entry: 'src/index.ts',
```

```
      name: 'MyLib',
      formats: ['es', 'umd', 'cjs'],
      fileName: (format) => `my-lib.${format}.js`,
    },

    // 写入磁盘（SSR 用）
    write: true,

    // 是否生成 manifest.json
    manifest: false,

    // SSR 构建
    ssr: false,

    // 报告压缩后大小（会变慢）
    reportCompressedSize: true,
  },
}

```

面试考点 — 分包策略：

```
// 方式一：对象形式（明确指定）
manualChunks: {
  'vue-vendor': ['vue', 'vue-router', 'pinia'],
  'ui-vendor': ['element-plus'],
}

// 方式二：函数形式（按规则分包）
manualChunks(id) {
  if (id.includes('node_modules')) {
    // 按包名分包
    const name = id.split('node_modules/')[1].split('/')[0]
    if (['vue', 'vue-router', 'pinia'].includes(name)) {
      return 'vue-vendor'
    }
    return 'vendor'
  }
}
```

```

```

分包的意义：

- 利用浏览器缓存（第三方库不常变）
- 减小单个文件体积
- 并行加载提升速度

---

10. `preview` — 预览服务器配置

```
{
  preview: {
    host: '0.0.0.0',
    port: 4173,
    open: true,
    proxy: {
      // 同 server.proxy
    },
  },
}

```

- 用于 `vite preview` 命令
- 预览 `dist` 目录的生产构建产物

---

11. `optimizeDeps` — 依赖预构建

```
{
  optimizeDeps: {
    // 强制预构建
    include: [
      'vue',
      'vue-router',
      'pinia',
      'axios',
      'element-plus/es',
      'element-plus/es/components/button/style/css',
```

```
      'lodash-es',
    ],

    // 排除预构建
    exclude: ['your-local-package'],

    // 强制重新预构建
    force: true,

    // 预构建入口
    entries: ['./src/main.ts'],

    // esbuild 配置
    esbuildOptions: {
      plugins: [],
    },
  },
}

```

面试考点 — 为什么需要预构建：

1. **CommonJS → ESM 转换**：浏览器只支持 ESM，需要把 CJS 依赖转成 ESM
2. **合并小模块**：把多文件依赖合并成单文件，减少 HTTP 请求（如 `lodash-es` 有 600+ 文件）
3. **缓存**：预构建产物缓存在 `node_modules/.vite` ，提升二次启动速度

---

12. `ssr` — 服务端渲染配置

```
{
  ssr: {
    // 外部化依赖（SSR 时不打包）
    external: ['lodash'],

    // 强制打包
    noExternal: ['your-component-lib'],

    // 构建目标
    target: 'node',
  },
}
```

```

```

---

13. `worker` — Web Worker 配置

```
{
  worker: {
    format: 'es',        // 'es' | 'iife'
    plugins: () => [],
    rollupOptions: {},
  },
}

```

---

14. `esbuild` — ESBuild 配置

```
{
  esbuild: {
    // JSX 配置
    jsxFactory: 'h',
    jsxFragment: 'Fragment',
    jsxInject: `import React from 'react'`,

    // 移除代码
    drop: ['console', 'debugger'],     // 生产环境移除
    pure: ['console.log'],             // 标记为纯函数（可被 tree-shaking）

    // 目标环境
    target: 'es2020',

    // 保留法律注释
    legalComments: 'none',
  },
}
```

```

```

生产环境移除 console 的推荐写法：

```
export default defineConfig({ mode }) => ({
  esbuild: {
    drop: mode === 'production' ? ['console', 'debugger'] : [],
  },
})

```

---

15. `envDir` 与 `envPrefix` — 环境变量

```
{
  envDir: './',              // .env 文件目录
  envPrefix: 'VITE_',        // 暴露给客户端的变量前缀（默认 VITE_）
}

```

环境变量加载优先级（从低到高）：

1. `.env` — 所有环境
2. `.env.local` — 所有环境，git 忽略
3. `.env.[mode]` — 指定模式
4. `.env.[mode].local` — 指定模式，git 忽略

使用方式：

```
// .env.development
VITE_API_BASE=http://localhost:8080
VITE_APP_TITLE=开发环境

// 代码中使用
console.log(import.meta.env.VITE_API_BASE)
console.log(import.meta.env.MODE)        // 'development'
console.log(import.meta.env.DEV)         // true
console.log(import.meta.env.PROD)        // false
```

```

```

---

16. `json` — JSON 导入配置

```
{
  json: {
    namedExports: true,     // 允许具名导入 JSON 字段
    stringify: false,       // 是否序列化（禁用具名导入，但体积更小）
  },
}

```

```
// namedExports: true 时
import { version } from './package.json'

// stringify: true 时（更小体积）
import pkg from './package.json'

```

---

17. `logLevel` 与 `clearScreen` — 日志配置

```
{
  logLevel: 'info',         // 'info' | 'warn' | 'error' | 'silent'
  clearScreen: true,        // 启动时清屏
}

```

三、完整配置模板

```
import { defineConfig, loadEnv } from 'vite'
import vue from '@vitejs/plugin-vue'
import path from 'node:path'

export default defineConfig(({ command, mode }) => {
  const env = loadEnv(mode, process.cwd(), '')
  const isDev = command === 'serve'
  const isProd = mode === 'production'

  return {
    // 基础
    base: env.VITE_BASE_PATH || '/',

    // 插件
    plugins: [
      vue(),
    ],

    // 解析
    resolve: {
      alias: {
        '@': path.resolve(__dirname, 'src'),
      },
    },

    // CSS
    css: {
      preprocessorOptions: {
        scss: {
          additionalData: `@use "@/styles/variables.scss" as *;`,
        },
      },
    },

    // 开发服务器
    server: {
      host: '0.0.0.0',
      port: 5173,
      open: true,
      proxy: {
        '/api': {
```

```
42          target: env.VITE_API_BASE,
43          changeOrigin: true,
44          rewrite: (path) => path.replace(/^\/api/, ''),
45        },
46      },
47    },
48
49    // 构建
50    build: {
51      outDir: 'dist',
52      sourcemap: !isProd,
53      minify: 'esbuild',
54      rollupOptions: {
55        output: {
56          manualChunks: {
57            'vendor-vue': ['vue', 'vue-router', 'pinia'],
58          },
59        },
60      },
61    },
62
63    // 预构建
64    optimizeDeps: {
65      include: ['vue', 'vue-router', 'pinia'],
66    },
67
68    // esbuild
69    esbuild: {
70      drop: isProd ? ['console', 'debugger'] : [],
71    },
72
73    // 全局常量
74    define: {
75      __APP_VERSION__: JSON.stringify(process.env.npm_package_version),
76    },
77  }
78 })
79
```

四、面试高频考点总结

考点	关键词
Vite 为什么快?	ESM、esbuild 预构建、按需编译、HMR
`base` 的作用?	部署子路径、CDN 地址
如何配置代理?	`server.proxy`、`changeOrigin`、`rewrite`
如何配置别名?	`resolve.alias`、配合 `tsconfig.json` 的 `paths`
如何分包?	`build.rollupOptions.output.manualChunks`
如何注入全局样式?	`css.preprocessorOptions.scss.additionalData`
依赖预构建是什么?	CJS→ESM、合并模块、`.vite` 缓存
环境变量怎么用?	`.env.*` 文件、`VITE_` 前缀、`import.meta.env`
如何移除 console?	`esbuild.drop` 或 `terserOptions.compress.drop_console`
`define` 的作用?	编译时全局常量替换